Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt 1: Klasyfikacja i regresja z użyciem perceptronu

Przedmiot: Sieci Neuronowe

Autorzy:
Jan Najowski
Stefan Stawicki

Prowadzący:
[Tytuł / Stopień, Imię i Nazwisko]

Warszawa, 19 października 2026

Spis treści

1 Wstęp i cel projektu **1**

2 Implementacja wielowarstwowego perceptronu (MLP) **1**

3 Eksperymenty — Zagadnienie regresji **2**
3.1 Zbiory danych i metodyka badawcza 2
3.2 Wpływ funkcji aktywacji w warstwach ukrytych 2
3.3 Wpływ architektury sieci: głębokość i szerokość 2
3.4 Wpływ miary błędu (funkcji straty) 2
3.5 Wpływ rozmiaru zbioru danych (opcjonalnie) 2
3.6 Podsumowanie i wnioski z części regresyjnej 2

4 Eksperymenty — Zagadnienie klasyfikacji **2**
4.1 Zbiory danych 2
4.2 Eksperyment 1 - Wpływ funkcji aktywacji w warstwach ukrytych 3
4.2.1 Zbiór danych simple 3
4.2.2 Zbiór danych three_gauss 5
4.3 Wpływ architektury sieci: głębokość i szerokość 8
4.4 Wpływ miary błędu (funkcji straty) 8
4.5 Podsumowanie i wnioski z części klasyfikacyjnej 8

5 Podsumowanie ogólne i wnioski końcowe **8**

1 Wstęp i cel projektu

Celem niniejszego projektu jest samodzielna implementacja wielowarstwowego perceptronu (*Multilayer Perceptron*, MLP) od podstaw z wykorzystaniem wyłącznie biblioteki `NumPy`. W ramach implementacji, przygotowano algorytm wstecznej propagacji błędu oraz stochastycznego spadku gradientu (*Stochastic Gradient Descent*). Przygotowany model posłużył do rozwiązania zagadnień aproksymacji nieliniowej dla funkcji $\mathbb{R} \to \mathbb{R}$ (zadanie regresji) oraz przydzielania klas punktom w przestrzeni $\mathbb{R}^2$ (zadanie klasyfikacji binarnej i wieloklasowej). W ramach pracy zbadano kilka aspektów wpływających na działanie sieci, takich jak dobór funkcji aktywacji (Sigmoid, Tanh, ReLU), głębokości i szerokości architektury (od 0 do 4 warstw ukrytych) oraz zastosowanych miar błędu na wyjściu modelu (MSE, BCE, CCE).

2 Implementacja wielowarstwowego perceptronu (MLP)

Kod źródłowy został podzielony na następujące moduły:

- **layers** — definiuje klasę bazową warstwy oraz warstwę w pełni połączoną (`Dense`), odpowiedzialną za transformację liniową, przechowywanie wag i biasów, a także inicjalizację wag przy pomocy zdefiniowanych rozkładów [1].
- **activations** — zawiera implementacje funkcji aktywacji (`ReLU`, `Sigmoid`, `Tanh`, i `Softmax`) wraz z metodami wyznaczania ich pochodnych na potrzeby propagacji wstecznej.
- **losses** — zawiera implementacje funkcji straty: błąd średniokwadratowy (`MSE`), binarna (`BCE`) i kategoryczna entropia krzyżowa (`CCE`).
- **optimizers** — implementuje algorytm *Stochastic gradient descent*.
- **mlp** — zamyka funckjonalność sieci w klasie `MLP`, przechowując listę warstw i implementując: propagację w przód, propagację wsteczną oraz kroki optymalizatora.
- **utils** — zawiera funkcje pomocnicze, w tym tworzenie mini batch'y, narzędzia do kodowania one-hot, metryki ewaluacyjne oraz ustawianie globalnego ziarna losowości.

Proces uczenia modelu sterowany jest przez metodę `MLP.fit()`, która w trakcie iteracji po kolejnych mini batchach i epokach na bieżąco zapisuje postępy uczenia. Wszystkie metadane zbierane są w formie słownika. Wśród jego kluczy znajdują się m.in. funkcje straty (w skali poszczególnych mini batchy oraz średnie na epokę dla zbioru treningowego i walidacyjnego) wprowadzone metryki (np. *Accuracy* dla klasyfikacji), normy Frobeniusa macierzy wag (dla wizualizacji postępów uczenia) oraz zmiany wartości kilku wybranych z góry wag sieci.

3 Eksperymenty — Zagadnienie regresji

3.1 Zbiory danych i metodyka badawcza

3.2 Wpływ funkcji aktywacji w warstwach ukrytych

3.3 Wpływ architektury sieci: głębokość i szerokość

3.4 Wpływ miary błędu (funkcji straty)

3.5 Wpływ rozmiaru zbioru danych (opcjonalnie)

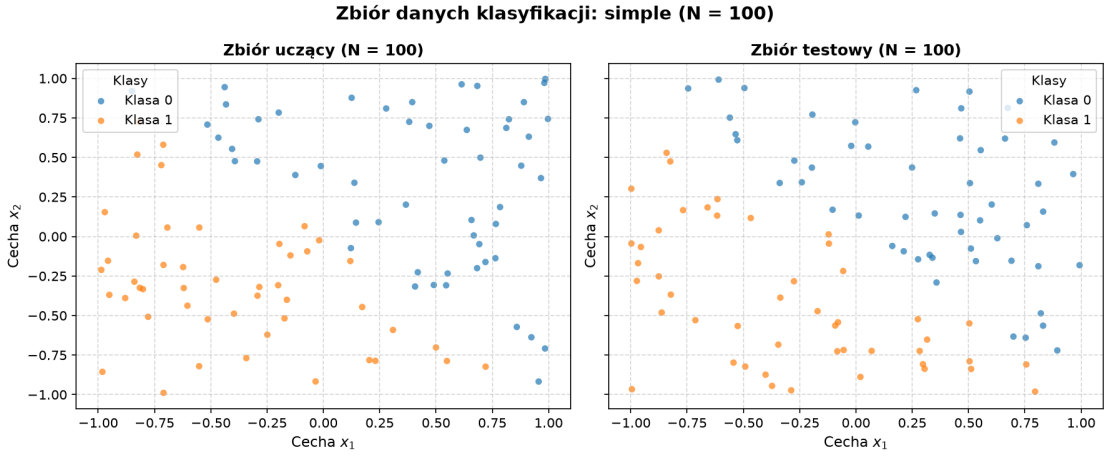
3.6 Podsumowanie i wnioski z części regresyjnej

4 Eksperymenty — Zagadnienie klasyfikacji

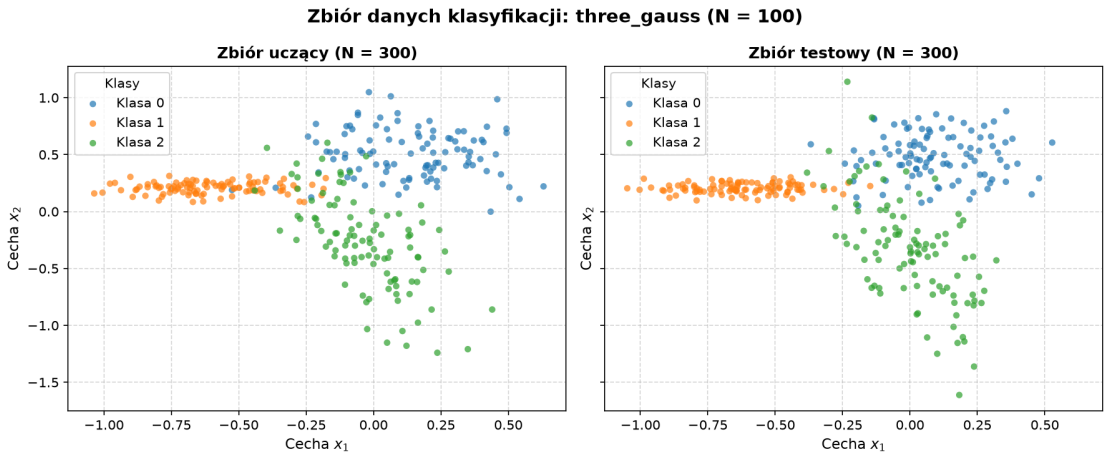
4.1 Zbiory danych

W tej części projektu, badano wykorzystanie modelu `MLP` do klasyfikacji punktów w przestzeni $\mathbb{R}^2$. Do zadania dostarczono dwa zbiory danych, każdy w kilku wersjach różniących się wielkością. Aby uwypuklić różnice między różnymi architekturami, skupiono się na rozmiarze $N = 100$.

- **Zbiór `data.simple`** — klasyfikacja binarnej (2 klasy). Skupiska klas rozdzielone są łagodną, ale nieliniową granicą (Rysunek 1). W warstwie wyjściowej stosowano przeważnie pojedynczy neuron z funkcją aktywacji `Sigmoid` oraz binarną entropię krzyżową (`BinaryCrossEntropyLoss`). Wyjątkiem jest eksperyment 3 którego celem było porównanie entropii krzyżowej do błędu średniokwadratowego (`MSE`).
- **Zbiór `data.three_gauss`** — klasyfikacja wieloklasowa (3 klasy). Skupiska powstały przez nałożenie na siebie dwuwymiarowych rozkładów gaussowskich (Rysunek 2). Kluczową cechą tego zbioru jest wyraźne nakładanie się rozkładów klas na środku przestrzeni. Ze względu na to przenikanie się klas, zbiór jest trudniejszy w klasyfikacji przez co idealne rozpoznawanie klas nie jest możliwe bez przeuczenia. Na wyjściu sieci stosowano 3 neurony z aktywacją `Softmax`, etykiety kodowane w postaci one-hot oraz funkcję kategorycznej entropii krzyżowej (`CategoricalCrossEntropyLoss`). Wyjątkiem ponownie był eksperyment 3 w którym porównano entropię krzyżową z `MSE`.



Rysunek 1: Wizualizacja zbioru uczącego i testowego dla problemu binarnego `data.simple` ($N = 100$).



Rysunek 2: Zbiór uczący i testowy dla problemu wieloklasowego `data.three_gauss` ($N = 100$ na klasę). Widoczne nakładanie się klas w centrum przestrzeni.

4.2 Eksperyment 1 - Wpływ funkcji aktywacji w warstwach ukrytych

W ramach eksperymentu zbadano wpływ doboru funkcji aktywacji w warstwach ukrytych (`Sigmoid`, `Tanh`, `ReLU`) na przebieg uczenia, stabilność wag oraz finalny przebieg granic decyzyjnych. Eksperymenty przeprowadzono na jednej architekturze $[2, 32, 16, C]$ ($C = 1$ dla klasyfikacji binarnej, $C = 3$ dla wieloklasowej) przy użyciu optymalizatora `SGD` ($\eta = 0.05$, 150 epok) dla 3 różnych ziaren losowości $(42, 123, 999)$.

Podsumowanie uzyskanych dokładności (*accuracy*) predykcji na zbiorze uczącym i testowym przedstawiono w Tabeli 1.

Tabela 1: Dokładność klasyfikacji dla modeli finalnych w zależności od funkcji aktywacji (średnia $\pm$ odchylenie standardowe z 3 ziaren).

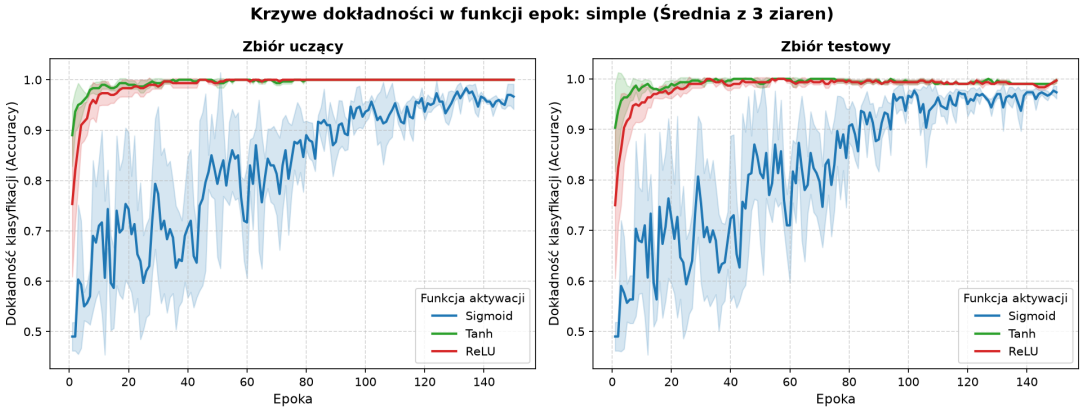
Zbiór danych	Funkcja aktywacji	Zbiór testowy [%]	Zbiór uczący [%]
`data.simple`	Sigmoid	97.33 ± 1.25	96.67 ± 2.49
	Tanh	99.67 ± 0.47	100.00 ± 0.00
	ReLU	99.67 ± 0.47	100.00 ± 0.00
`data.three_gauss`	Sigmoid	93.11 ± 0.16	92.44 ± 0.16
	Tanh	92.89 ± 0.16	92.56 ± 0.16
	ReLU	93.22 ± 0.16	94.11 ± 0.16

4.2.1 Zbiór danych simple

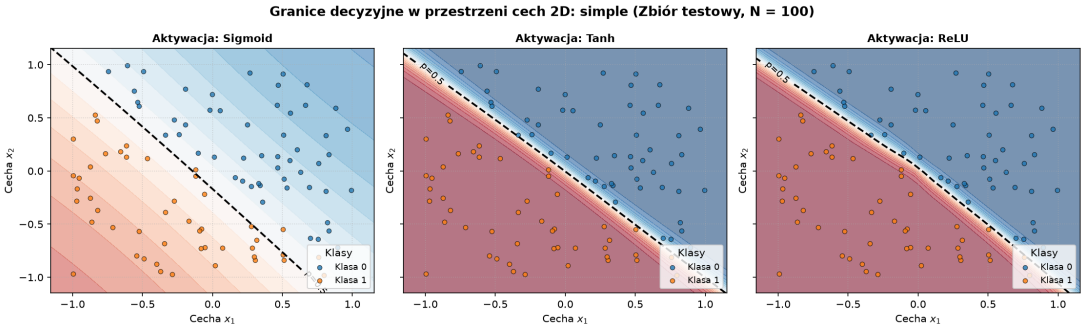
W zadaniu binarnym modele z aktywacją `Tanh` oraz `ReLU` zdołały w 100% nauczyć się rozpoznawać klasy na zbiorach treningowych i dobrze generalizują to na zbiorze testowym (99.67%). Funkcja `Sigmoid` zbiegała wyraźnie wolniej ze względu na nasycenie gradientu (stosunkowo mała wartość pochodnej funkcji sigma). Poniżej przedstawiono wykresy ilustrujące przebieg uczenia i finalne rezultaty na zbiorze prostym.

Na Rysunku 3 można zaobserwować, że dokładność sieci z aktywacją `Sigmoid` znacznie wolniej zbiega do 1. Dodatkowo trajektoria krzywej jest bardziej chaotyczna i wykazuje większe odchylenie standardowe (na badanej próbce 3 ziaren losowych), co sugeruje, że aktywacja `Sigmoid` jest bardziej wrażliwa na losową inicjalizację wag. Gdy porównać nasycenie kolorów na Rysunku 4, widać, że granice decyzyjne dla funkcji `ReLU` i `Tanh` są jasno zdefiniowane, co oznacza, że model jest pewny swoich decyzji. W przypadku `Sigmoid` sytuacja jest odwrotna — występuje szerokie

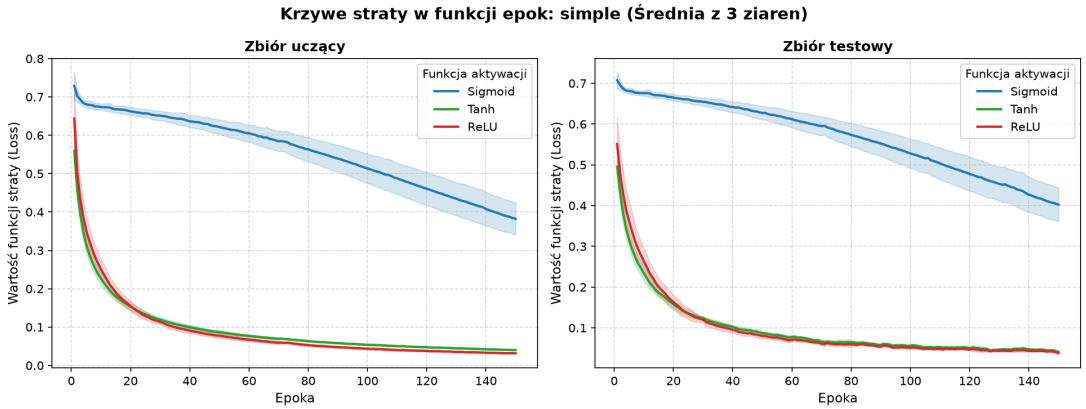
pasmo wysokiej niepewności (szacowane prawdopodobieństwo bliskie 0.5). Przebieg funkcji straty (Rysunek 5) pokazuje gwałtowny spadek błędu treningowego dla `ReLU` i `Tanh` już w pierwszych 20 epokach, podczas gdy strata w modelu `Sigmoid` maleje powoli i jednostajnie przez cały trening. Ewolucja pierwszej wagi $w_{0,0}$ oraz biasu b_0 pierwszego neuronu (Rysunek 6) pokazuje stabilny ruch parametrów bez oscylacji, z największą dynamiką adaptacji widoczną w warstwie wyjściowej. Z kolei zestawienie norm Frobeniusa macierzy wag w każdej z warstw (Rysunek 7) wskazuje na powolny, wzrost normy wag we wszystkich warstwach bez nagłych nieprzewidywalnych zmian, przy czym najwyższe wartości osiąga sieć z aktywacją `ReLU`.



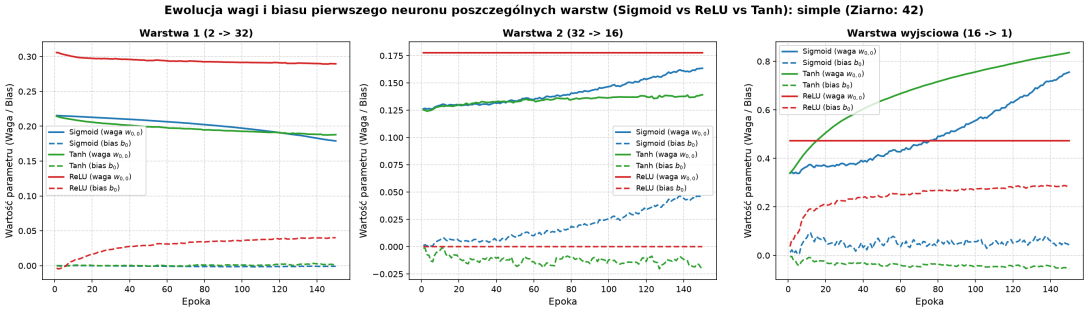
Rysunek 3: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.simple` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



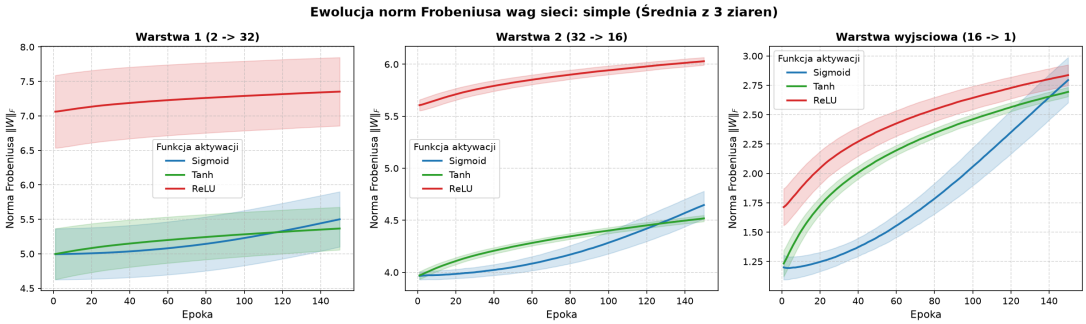
Rysunek 4: Granice decyzyjne w przestrzeni $\mathbb{R}^2$ dla zbioru `data.simple` w zależności od funkcji aktywacji.



Rysunek 5: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.simple` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 6: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu poszczególnych warstw dla zbioru `data.simple` (ziarno 42).



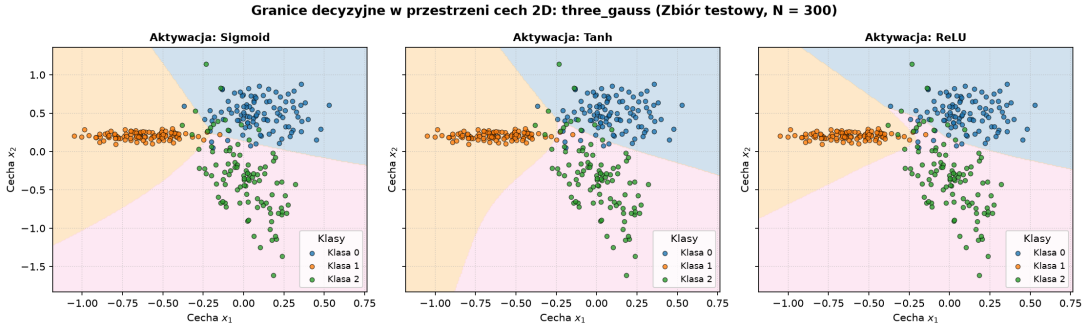
Rysunek 7: Ewolucja norm Frobeniusa wag poszczególnych warstw w trakcie treningu dla zbioru `data.simple`.

4.2.2 Zbiór danych three_gauss

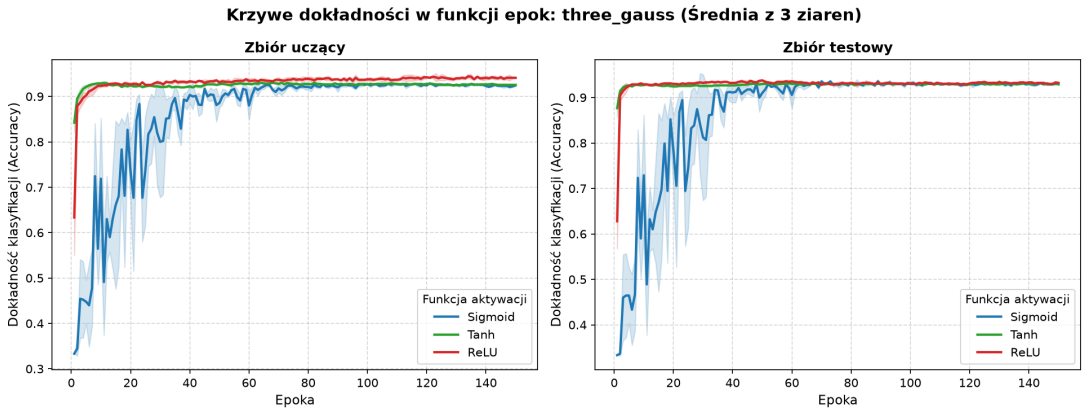
W zagadnieniu wieloklasowym wszystkie badane funkcje aktywacji osiągnęły zbliżoną dokładność na poziomie 92.9–93.2%. Ograniczenie to wynika z fizycznego nakładania się na siebie rozkładów trzech klas.

Na Rysunku 8 przedstawiono wyznaczone granice decyzyjne w przestrzeni $\mathbb{R}^2$. Funkcja `ReLU` tworzy odcinkowo liniowe krawędzie podziału, podczas gdy `Tanh` i `Sigmoid` wyznaczają gładkie krzywe. We wszystkich modelach widoczne są punkty błędnie zaklasyfikowane w centralnej strefie nakładania się klas. Krzywe dokładności na Rysunku 9 pokazują, że `ReLU` i `Tanh` już w kilkunastu epokach osiągają dokładność ok. 93%, podczas gdy `Sigmoid` początkowo mocno oscyluje i

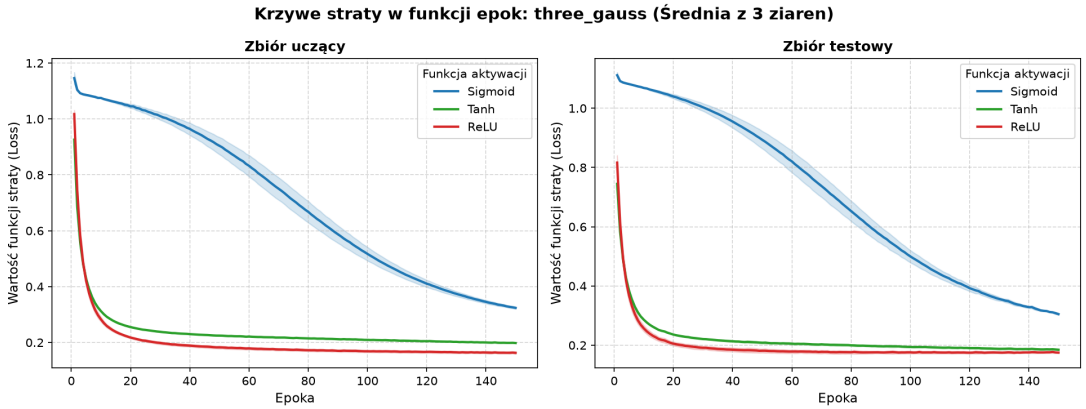
potrzebuje ponad 80 epok na ustabilizowanie predykcji. Przebieg funkcji straty kategorycznej entropii krzyżowej (`CCE`) (Rysunek 10) potwierdza szybki spadek błędu dla `ReLU` i `Tanh` poniżej poziomu 0.2, podczas gdy strata `Sigmoid` zbiega powoli, osiągając ostatecznie wartość ok. 0.3. Analiza trajektorii wag $w_{0,0}$ i biasów b_0 pierwszego neuronu (Rysunek 11) wskazuje na płynną ewolucję parametrów poszczególnych warstw, które po początkowym etapie uczenia zbiegają do pewnych wartości. Wykres norm Frobeniusa macierzy wag (Rysunek 12) świadczy o stabilności procesu uczenia we wszystkich warstwach sieci, potwierdzając brak problemów z eksplozją gradientu przy zastosowanym kroku uczenia $\eta = 0.05$.



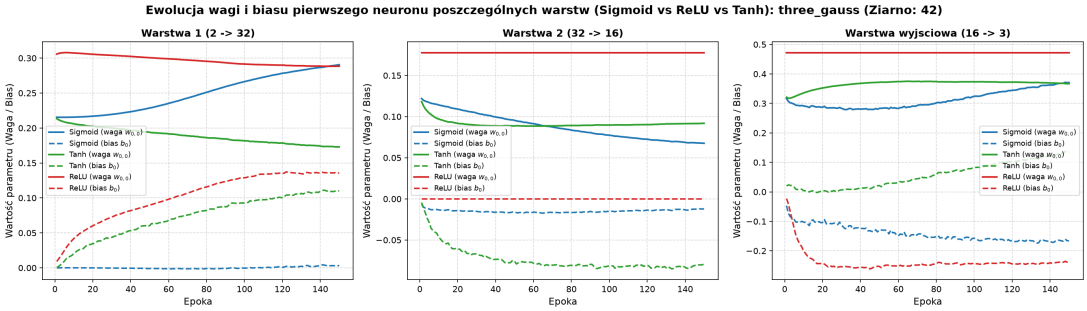
Rysunek 8: Granice decyzyjne w przestrzeni $\mathbb{R}^2$ dla zbioru `data.three_gauss` w zależności od funkcji aktywacji.



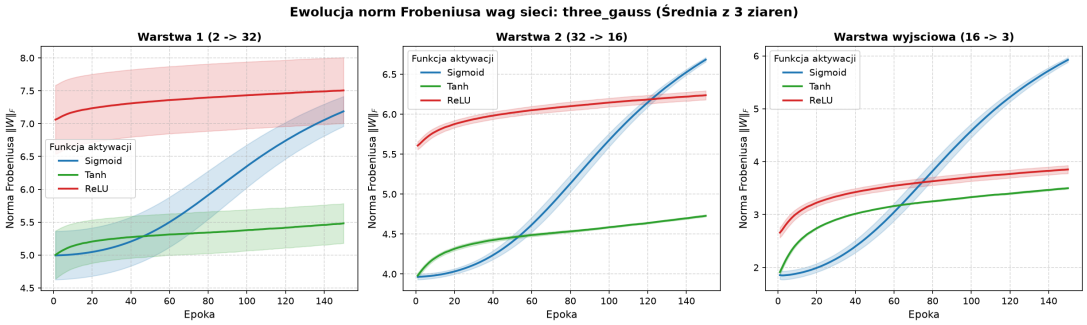
Rysunek 9: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 10: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 11: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu poszczególnych warstw dla zbioru `data.three_gauss` (ziarno 42).



Rysunek 12: Ewolucja norm Frobeniusa wag poszczególnych warstw w trakcie treningu dla zbioru `data.three_gauss`.

Wnioski. Podsumowując wyniki:

- Funkcje aktywacji `Tanh` oraz `ReLU` zapewniają znacznie szybszą redukcję błędu i wyższą odporność na losową inicjalizację parametrów niż funkcja `Sigmoid`, która cierpi na spowolnienie uczenia wywołane nasyceniem gradientu.
- Kształt wyznaczonych granic decyzyjnych odzwierciedla charakter funkcji aktywacji (odcinkowo liniowe dla `ReLU` vs gładkie dla `Tanh`). W zbiorze `data.three_gauss` w którym klasy się nakładają wszystkie modele osiągają zbliżoną skuteczność ograniczoną górną granicą ok. 93%.

- Ewolucja norm wag oraz trajektorii pojedynczych neuronów przebiega stabilnie we wszystkich wariantach, potwierdzając że implementacja jest poprawna.

4.3 Wpływ architektury sieci: głębokość i szerokość

4.4 Wpływ miary błędu (funkcji straty)

4.5 Podsumowanie i wnioski z części klasyfikacyjnej

5 Podsumowanie ogólne i wnioski końcowe

Literatura

[1] The Deep Hub, *He and Xavier Weight Initialization Functions*, https://medium.com/thedeephub/he-and-xavier-weight-initialization-functions-acedc5322ce5.